

LAFSys Development Roadmap

Complete Structural Guide for GUI Implementation

Commencing: September 3rd, 2026

Phase 1: Project Initialization & UI Architecture

The foundation of the project. Before writing any Java code, the system architecture and visual layout must be planned.

- **Define Data Requirements:** Finalize the exact attributes an "Item" will hold (ID, Title, Description, Category, Date, Status, Poster Name, Location).
- **Wireframing:** Sketch the visual layout of the application. Design the layout for a single window containing a `JTabbedPane` with three tabs: *Post Item*, *View/Search*, and *Claim*.
- **Environment Setup:** Create the project folder structure. Open the Java IDE (Eclipse/IntelliJ/VS Code), create the project workspace, and establish a file-sharing protocol for code merging.

🎯 **Milestone Goal: Complete system blueprints and functional IDE environments.**

Phase 2: Core Data Architecture (Backend)

Building the internal data structures that will hold the application's information in memory.

- **Create Data Model:** Create `Item.java`. Implement Encapsulation by declaring all 8 item attributes as `private`.
- **Constructor & Methods:** Write a constructor that accepts all attributes. Generate standard `get` and `set` methods for every variable.
- **Data Formatting:** Override the standard `toString()` method to output a cleanly formatted String. This specific string format will be what users eventually see in the GUI text area.

🎯 **Milestone Goal: A fully functioning, encapsulated Item class ready to hold data.**

Phase 3: Business Logic Implementation (Backend)

Developing the engine of the application that manages data storage, retrieval, and modification.

- **Initialize Storage:** Create `LostAndFoundBoard.java`. Declare a private `ArrayList<Item>` to act as the runtime database.
- **Write Insert Logic:** Create an `addItem(Item item)` method to push new objects into the `ArrayList`.
- **Write Retrieval Logic:** Create methods to loop through the `ArrayList`, filter items by "Lost" or "Found" status, and append them to a `StringBuilder` to return one massive formatted `String`.
- **Write Search & Claim Logic:** Implement a keyword search method (using `.contains()`) and a claim method that searches for an ID and updates the item's status to "Claimed".

🎯 **Milestone Goal:** The backend system is completely capable of managing data, independent of any UI.

Phase 4: Graphical User Interface Construction (Frontend)

Translating the Phase 1 wireframes into actual Java Swing components.

- **Main Window Setup:** Create `MainGUI.java`. Initialize the primary `JFrame` (set size, title, close operation) and add a `JTabbedPane`.
- **Build Form Tab:** Construct a `JPanel` for posting items. Add `JLabel` and `TextField` pairs for inputs. Add a `JComboBox` for Lost/Found selection, and a "Submit" `Button`.
- **Build Display Tab:** Construct a `JPanel` featuring a search bar (`TextField`), action buttons (Search, View Lost, View Found), and a large `TextArea` (wrapped in a `JScrollPane`) for output.
- **Build Claim Tab:** Construct a simple `JPanel` with an ID input field and a "Claim Item" button.

🎯 **Milestone Goal:** The visual application opens perfectly, though buttons do not work yet.

Phase 5: Event Wiring & Integration

Connecting the graphical buttons (Phase 4) to the backend engine (Phase 3) using `ActionListeners`.

- **Instantiate Backend:** Create a global instance of `LostAndFoundBoard` inside `MainGUI.java` so all tabs share the same data list.
- **Wire Submit Button:** Add an `ActionListener` that triggers on click: extracts text using `getText()`, constructs a `new Item()`, calls `addItem()`, and triggers a `JOptionPane` success popup.
- **Wire View/Search Buttons:** Add listeners that call the backend retrieval methods and use `textArea.setText()` to display the returned data on the screen.
- **Wire Claim Button:** Add a listener to parse the inputted ID, pass it to the backend claim method, and display either a success or failure popup based on the boolean result.

🎯 **Milestone Goal:** A fully interactive, working GUI application.

Phase 6: Quality Assurance & Testing

Running the completed application through rigorous testing scenarios to ensure stability.

- **Functional Testing:** Input dummy data to ensure items are correctly posted, displayed, and categorized.
- **Edge Case Testing:** Attempt to search for keywords that don't exist, claim items that are already claimed, and input letters where numbers are expected (to test error handling).
- **UI/UX Review:** Ensure window components do not overlap improperly and that text areas scroll correctly when populated with large amounts of data.
- **Asset Capture:** Take high-resolution screenshots of every window, form, and popup for documentation purposes.

🎯 **Milestone Goal: A bug-free application with all visual assets captured.**

Phase 7: Academic Documentation & Delivery

Packaging the software development lifecycle into a professional academic report.

- **Structure the Report:** Create sections for Introduction, Problem Statement, System Design (Flowcharts/Class Diagrams), Implementation, and Conclusion.
- **Document the Code:** Copy the fully commented Java source files (Item, LostAndFoundBoard, MainGUI) into the Implementation section.
- **Integrate Screenshots:** Place the captured GUI screenshots into the report with proper Figure captions detailing the test cases they represent.
- **Final Polish:** Apply consistent typography, spacing, and pagination. Export the final document as a printable PDF.

🎯 **Milestone Goal: Project is 100% complete and ready for academic submission.**